

Logical Operators

Short-Circuiting

Logical operators `and` and `or` aren't like other binary operators

Because they **short-circuit**

If the left operand determines the *truth value* of the entire expression,

we **don't** evaluate the right operand

For example, in `false and doSomething()`, the function `doSomething()` should never execute

A New AST Node

Because of short-circuiting, we can't reuse our existing `Binary` expression class

Our `Binary` expression evaluates both of its operands *before* evaluating the operator itself

So, we need to introduce a new `Logical` *expression* node to handle this special behavior

```
1  tool/GenerateAst.java
2  in main()
```

```
1      "Grouping : Expr expression",
2      "Literal  : Object value",
3      "Logical   : Expr left, Token operator, Expr right",
4      "Unary     : Token operator, Expr right"
```

Updating the Grammar

We need to add logical operators to our grammar. They sit right above `equality` in precedence

And `or` has lower precedence than `and` (following `c`)

```
1 expression → assignment ;
2 assignment → IDENTIFIER "=" assignment | logic_or ;
3 logic_or    → logic_and ( "or" logic_and )* ;
4 logic_and   → equality ( "and" equality )* ;
```

Because of this, we need to update our `assignment` rule to call `or` instead of `equality`

```
1 lox/Parser.java
2 in assignment()
```

```
1 private Expr assignment() {
2     Expr expr = or();
3     // ...
```

Parsing OR

Let's implement the parsing logic for `or`

It looks very similar to our other binary operator parsing methods

```
1  lox/Parser.java
2  after assignment()
```

```
1  private Expr or() {
2      Expr expr = and();
3
4      while (match(OR)) {
5          Token operator = previous();
6          Expr right = and();
7          expr = new Expr.Logical(expr, operator, right);
8      }
9
10     return expr;
11 }
```

Parsing AND

Next, we implement the parsing logic for `and`

It relies on `equality()`, which is the next highest level of precedence

```
1  lox/Parser.java
2  after or()

1  private Expr and() {
2      Expr expr = equality();
3
4      while (match(AND)) {
5          Token operator = previous();
6          Expr right = equality();
7          expr = new Expr.Logical(expr, operator, right);
8      }
9
10     return expr;
11 }
```

Interpreting Logical Expressions

Now for the interpreter. We evaluate the left operand first

Then we look at its truthiness to see if we can short-circuit and skip the right side entirely

```
1  lox/Interpreter.java
2  in class Interpreter

1  @Override
2  public Object visitLogicalExpr(Expr.Logical expr) {
3      Object left = evaluate(expr.left);
4
5      if (expr.operator.type == TokenType.OR) {
6          if (isTruthy(left)) return left;
7      } else {
8          if (!isTruthy(left)) return left;
9      }
10
11     return evaluate(expr.right);
12 }
```

Returning Values

Notice that we return the **value** of the operand itself with `return left;`

We don't return a strict `true` or `false` boolean literal unless the expression evaluated to one

This is a common, powerful feature in dynamically typed languages like Ruby, JavaScript, and Python

```
1  print "hi" or 2; // "hi"  
2  print nil or "yes"; // "yes"
```